

Track A — Stage A (Detector) — Local Runner (guide)

This note explains what `local_extraction/trackA_stageA.py` does and how to use it. It is a simple, local-only runner for Track A / Stage A (the detector on the last frame), designed to read inputs from your local folders and write all outputs to a local run directory.

What it does

- Scans your extracted frames (default: `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg`).
- For each image (by default the last frame per `<uid>`), generates up to K detection proposals (candidate boxes).
- Saves results under a Track_A run folder:
 - `candidates.jsonl`: one JSON record per image with `{uid, frame, boxes: [{x1,y1,x2,y2,conf,cls}]}`
 - optional per-image CSVs with the same info (one file per image)
 - `summary.json` with counts, mode, K, paths, and runtime You can run proposals in two ways:
- Oracle mode (`DETECTION_MODE='oracle'`): uses your existing YOLO label files as the proposals (fastest way to dry-run the pipeline, useful for recall@K analysis).
- YOLO mode (`DETECTION_MODE='yolo'`): runs Ultralytics YOLO locally (no network). You must have the `ultralytics` package installed and a weights file on disk.

Where inputs come from

- Frames: `local_extraction/v2/extracted_frames/<uid>/<frame:07d>.jpg`
- Oracle labels (if using oracle mode):
`local_extraction/v2/yolo_labels_540/{clips|videos}/<uid>/<frame:07d>.txt` These are produced by your earlier extraction + label generation steps and live entirely in your local workspace.

Where outputs are written

- Run folder: `local_extraction/runs/Track_A/StageA_<RUN_NAME>/`
 - `candidates.jsonl`
 - `candidates/<uid>_<frame>.csv` (if `SAVE_PER_IMAGE_CSV=True`)
 - `summary.json` You can change `RUN_NAME` at the top of the script to separate different experiments (e.g., different K, different YOLO thresholds).

The toggles (edit in the file, no CLI needed)

- `RUN_NAME`: name of the current run; used to create the run folder.
- `VERSION`: subfolder under `local_extraction/` (default: `v2`).
- `SPACE`: `'clips'` or `'videos'`. Only affects the oracle label path.
- `FRAMES_ROOT`, `LABELS_ROOT`: where to read frames/labels from (local paths).
- `LAST_FRAME_ONLY`: `True` to process the last frame per `<uid>`; `False` to process all frames.
- `MAX_IMAGES`: cap the number of images processed (e.g., `500`) for quick tests.
- `DETECTION_MODE`: `'oracle'` or `'yolo'`.

- **K**: keep top-K candidates per image.
- YOLO-specific (only used if **DETECTION_MODE='yolo'**):
 - **YOLO_WEIGHTS**: path to a local weights file (e.g., **yolov8s.pt**).
 - **YOLO_IMGSZ**: square inference size (e.g., **640**).
 - **YOLO_CONF**: confidence threshold.
 - **YOLO_IOU**: NMS IoU threshold.
- **SAVE_PER_IMAGE_CSV**: **True** to also write a small CSV per image.
- **RUN_DIR**: full path of the run directory (auto-built from **RUN_NAME**).

How it works inside

1. Build the run directory (**ensure_run_dir**).
2. Collect images from **FRAMES_ROOT**:
 - By default, it yields the last ***.jpg** file inside each **<uid>** folder.
 - If **LAST_FRAME_ONLY=False**, it yields all frames.
3. For each image:
 - If **DETECTION_MODE='oracle'**: reads the label txt file, converts normalized YOLO **cx,cy,w,h** back to pixel **x1,y1,x2,y2** using the image size, then keeps top-K (or all if fewer).
 - If **DETECTION_MODE='yolo'**: runs Ultralytics YOLO with your local weights and thresholds, then keeps top-K by confidence.
 - Writes one JSON line with **{uid, frame, boxes:[...]}** into **candidates.jsonl**.
 - Optionally writes a per-image CSV for quick viewing.
4. After all images:
 - Writes **summary.json** with image count, average boxes per image, settings, and runtime.

File formats

- **candidates.jsonl** (one record per line):

```
{"uid":"00be...1705","frame":99,"boxes":
[{"x1":161.8,"y1":-0.3,"x2":373.9,"y2":244.8,"conf":1.0,"cls":26}]}
```

- per-image CSV (optional):

```
x1,y1,x2,y2,conf,cls
161.8,-0.3,373.9,244.8,1.0000,26
```

Typical workflows

- Dry-run with oracle proposals (fast, no heavy models):
 - Set **DETECTION_MODE='oracle'**, **LAST_FRAME_ONLY=True**, pick a small **K** (e.g., 5) and an informative **RUN_NAME**.
 - Run: **python local_extraction/trackA_stageA.py**
 - Inspect **summary.json** and **candidates.jsonl**.
- Switch to YOLO proposals (if you have weights installed locally):

- Set `DETECTION_MODE='yolo'` and adjust `YOLO_WEIGHTS`, `YOLO_CONF`, `YOLO_IOU`.
- Run the script again and compare summaries.

Tips

- Keep runs short (use `MAX_IMAGES` and last-frame processing) and save outputs frequently in `local_extraction/runs`.
- The oracle mode is useful for recall@K checks: if you convert GT to proposals, you should see near-perfect recall with small K.
- For broader experiments, duplicate the script or change `RUN_NAME` to keep logs tidy.